

# Helix VPN -- Phase 2 (MVP2) Client Applications -- Comprehensive Specification

## Helix VPN -- Phase 2 (MVP2) Client Applications -- Comprehensive Specification

**Revision: 2 Last modified:** 2026-07-04T12:00:00Z

| Field          | Value                                                                                   |
|----------------|-----------------------------------------------------------------------------------------|
| Version        | 1.1                                                                                     |
| Date           | 2026-07-03                                                                              |
| Status         | Draft for Review                                                                        |
| Classification | Internal -- Engineering Confidential                                                    |
| Author         | Technical Architecture Team                                                             |
| Audience       | Executive Leadership, Engineering Leads,<br>Platform Teams, DevOps, Security Review, QA |
| Next Review    | 2026-07-17                                                                              |

**Revision 2 changelog (2026-07-04):** Deep gap-analysis + production-readiness hardening pass across the full MVP2 corpus. Fixed the stale §8.1 document registry (it referenced never-delivered filenames), reconciled the best-case/expected/worst-case timeline against MVP2\_IMPLEMENTATION\_ROADMAP.md, and added explicit cross-references to the new Enterprise Hardening sections added to every platform document. See each linked document's own revision header for its specific changes.

### Table of Contents

- 1. Document Information

- 2. [Executive Summary](#)
- 3. [Project Context](#)
- 4. [Platform Coverage Matrix](#)
- 5. [Architecture at a Glance](#)
- 6. [Technology Stack Summary](#)
- 7. [Key Features](#)
- 8. [Documentation Structure](#)
- 9. [Success Criteria](#)
- 10. [Risk Summary](#)
- 11. [Glossary](#)

---

## 2. Executive Summary

### 2.1 What MVP2 Covers

MVP2 represents the most ambitious expansion in the Helix VPN product roadmap: the development of **complete, production-grade client applications for eight distinct target platforms across seven operating systems**. Where MVP1 established the foundational VPN server infrastructure, authentication systems, and administrative backends, MVP2 delivers the end-user experience -- the actual applications that customers will install, configure, and use to protect their network traffic across every major consumer and enterprise computing platform.

This phase is not a simple port of a single codebase to multiple targets. It is a **deliberately architected, multi-platform ecosystem** built around a shared Rust core (helix-core) that encapsulates all VPN protocol logic, cryptographic operations, and network management behind a unified API. Each platform implements only a thin native UI layer and a platform-specific adapter for OS VPN APIs, achieving industry-leading code reuse while preserving native performance, native look-and-feel, and deep OS integration on every target.

### 2.2 Target Platforms at a Glance

MVP2 targets **eight platforms** organized across four priority tiers:

|                                            |                                                 |
|--------------------------------------------|-------------------------------------------------|
| TIER 1 (P0 -- Core Revenue Platforms)      | : 4 platforms -- macOS, Windows, Linux, Android |
| TIER 2 (P1 -- Strategic Growth Platforms)  | : 2 platforms -- iOS, HarmonyOS                 |
| TIER 3 (P2 -- Specialized/Niche Platforms) | : 1 platform -- Aurora OS                       |
| TIER 4 (P3 -- Companion/Extension)         | : 1 platform -- Web Browser Extension           |

-----  
TOTAL : 8 platforms  
across 7 operating systems

## 2.3 Strategic Goals

MVP2 is governed by three non-negotiable architectural pillars:

| Pillar                    | Target                                                                   | Rationale                                                                                   |
|---------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Maximal Code Reuse</b> | 70-85% shared code across all platforms                                  | Reduces maintenance burden, eliminates behavior divergence, accelerates feature delivery    |
| <b>Minimal Footprint</b>  | Sub-25 MB bundles on all platforms; sub-15 MB on desktop                 | Security-conscious users scrutinize application size; smaller bundles reduce attack surface |
| <b>Native Performance</b> | <1s connection time (desktop), <2s (mobile); <100ms kill switch response | Users expect instant-on protection; kill switch latency is a security-critical metric       |

## 2.4 Estimated Scope and Timeline

| Dimension                                          | Estimate                                                                                                                                       |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Total Development Duration</b>                  | 30 weeks best case / <b>36 weeks expected (planning baseline)</b> / 44 weeks worst case, across 9 implementation phases                        |
| <b>Estimated Lines of Code (Rust Core)</b>         | ~23,000 lines (shared across all platforms)                                                                                                    |
| <b>Estimated Lines of Code (Platform-Specific)</b> | ~6,500 lines (UI + adapters combined)                                                                                                          |
| <b>Shared Code Ratio</b>                           | 70-85% depending on platform                                                                                                                   |
| <b>Team Size</b>                                   | 8-12 engineers (11 is the planning-baseline headcount: 2 core Rust, 2 desktop, 3 mobile, 1 Aurora/Qt, 1 web, 1 DevOps, 1 floating QA/security) |
| <b>CI/CD Targets</b>                               | 14+ build artifacts per release cycle                                                                                                          |

**Reconciliation note (Revision 2):** earlier drafts of this table and of MVP2\_ARCHITECTURE.md §7.1 quoted an unbuffered **30-week** figure taken directly from summing each phase's nominal duration with no schedule risk applied.

MVP2\_IMPLEMENTATION\_ROADMAP.md §13 ("Risk-Adjusted Timeline") is the authoritative, detailed, per-phase schedule: it models the same 30-week figure as the **20%-probability best case**, and derives a **36-week expected case (60% probability)** and a **44-week worst case (20% probability)** by adding phase-specific contingency buffers (entitlement review delays, store review latency, cross-compilation toolchain friction). This document and README.md now use the 36-week **expected/planning** figure for staffing and stakeholder commitments, while still surfacing the 30-week best case as the optimistic floor. Treat MVP2\_IMPLEMENTATION\_ROADMAP.md §11-§13 as the single source of truth for schedule and headcount; this table is a summary only.

---

## 3. Project Context

### 3.1 Helix VPN Project Overview

Helix VPN is a next-generation virtual private network service designed for maximum privacy, security, and cross-platform ubiquity. The product targets both consumer and enterprise markets with a differentiated value proposition built on three pillars: **memory-safe implementation** (Rust throughout), **minimal resource footprint** (sub-25 MB clients), and **uncompromising multi-platform coverage** (8 platforms from a single codebase).

The project is structured in phased deliverables:

|                                                         |                 |
|---------------------------------------------------------|-----------------|
| Phase 1 (MVP1): Server Infrastructure & Admin Backend   | [COMPLETED]     |
| Phase 2 (MVP2): Client Applications (8 platforms)       | [THIS DOCUMENT] |
| Phase 3 (MVP3): Enterprise Features & Ecosystem         | [PLANNED]       |
| Phase 4 (MVP4): Advanced Anti-Censorship & Post-Quantum | [PLANNED]       |

**Flagged inconsistency (found 2026-07-04, NOT silently corrected here — see docs/research/UNIFIED\_PHASE\_ROADMAP.md §4.2 "Narrative B" + open decision R-2).** The "[COMPLETED]" framing of Phase 1/MVP1 below — including a "globally distributed WireGuard server fleet" and "[COMPLETED]" billing integration — does **not** match the actual, current Phase 0/1 specification in docs/research/mvp/final/SPECIFICATION.md §8.1: a single self-hostable control-plane MVP, not yet built, with billing explicitly out of scope until the Phase-3 "billing-optional" epic (HVPN-P3-E24). This reads as carried-over boilerplate from a generic SaaS-VPN framing rather than a description of *this* project's actual state. It is intentionally left as-is here (correcting the narrative is a product-framing decision for the operator, not

something a documentation-hardening pass should do unilaterally — §11.4.35) and is tracked as an open item at UNIFIED\_PHASE\_ROADMAP.md R-2.

## 3.2 MVP1 Scope (Completed Deliverables)

MVP1 established the server-side foundation upon which all client applications depend:

- **VPN Server Network:** Globally distributed WireGuard server fleet with automated provisioning
- **Authentication Service:** Multi-factor authentication, OAuth2/OIDC integration, account management
- **Client API:** RESTful API for server list retrieval, latency testing, and connection credential distribution
- **Admin Dashboard:** Web-based management interface for server monitoring, user management, and analytics
- **Billing Integration:** Subscription management, payment processing, usage tracking
- **Utils Service:** Shared utilities including logging, metrics, configuration management

The MVP1 deliverables provide the **backend contract** that all MVP2 client applications consume. The client API defines the interface for server discovery, the authentication service provides the identity layer, and the admin dashboard offers visibility into the client fleet.

## 3.3 MVP2 Scope Expansion

MVP2 transforms Helix VPN from a server-side product into a **complete end-to-end solution**. The scope encompasses:

### Core Development:

- Design and implementation of helix-core -- the Rust shared library powering all platforms
- WireGuard protocol implementation (via boringtun-derived codebase)
- Shadowsocks obfuscation layer (SIP022 AEAD-2022)
- MASQUE/QUIC transport protocol (RFC 9298)
- Platform abstraction layer with adapter trait system
- Cross-compilation pipeline for 14+ target architectures

### Desktop Applications (3 platforms):

- helix-desktop for macOS 12+ (Apple Silicon + Intel)
- helix-desktop for Windows 10/11 (x64 + ARM64)
- helix-desktop for Linux (Ubuntu 22.04+, Fedora 39+, x64 + ARM64)

### Mobile Applications (3 platforms):

- `helix-mobile` for Android 8+ (API 26+, ARM64/x86\_64)
- `helix-mobile` for iOS 15+ (ARM64)
- `helix-mobile` for HarmonyOS (API 12+, via Flutter ohos embedding)

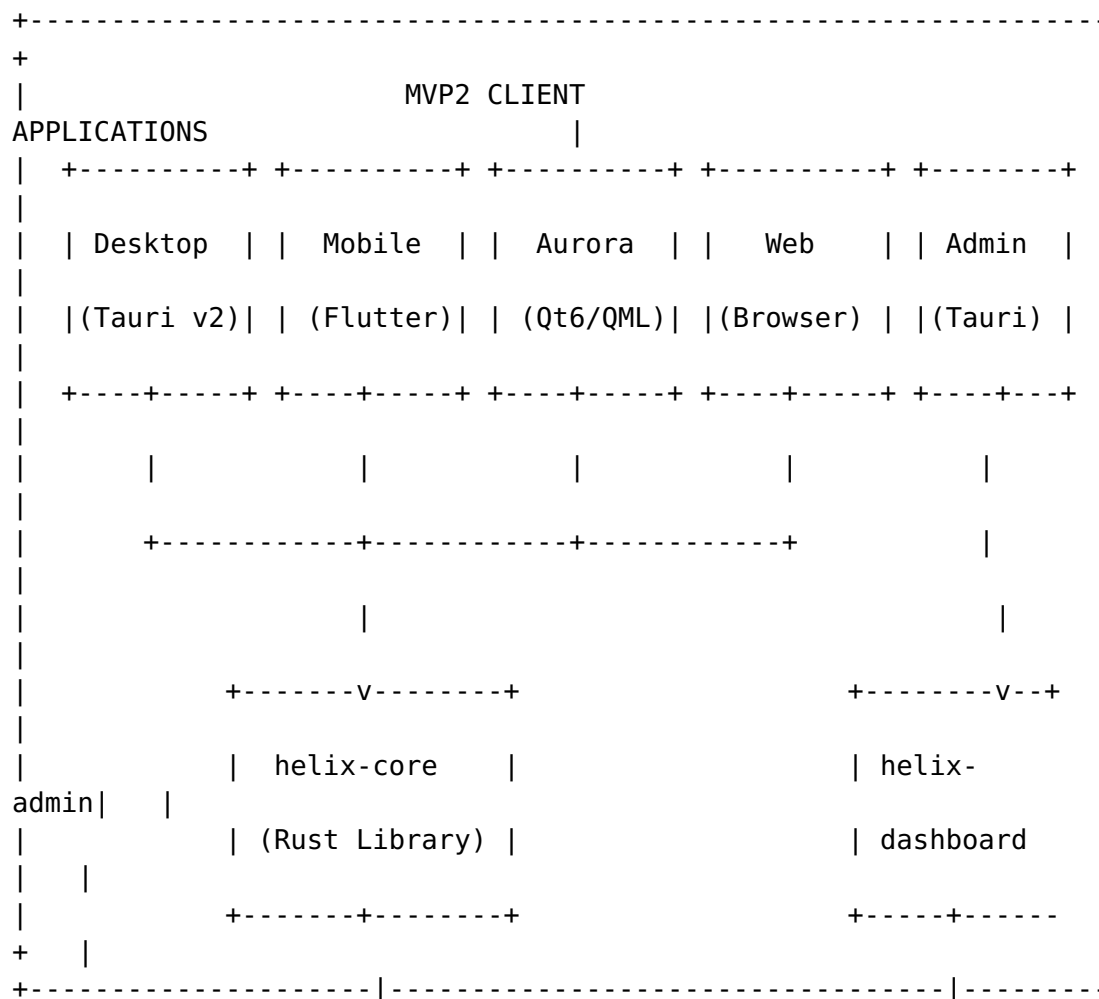
### Specialized Platforms (2 platforms):

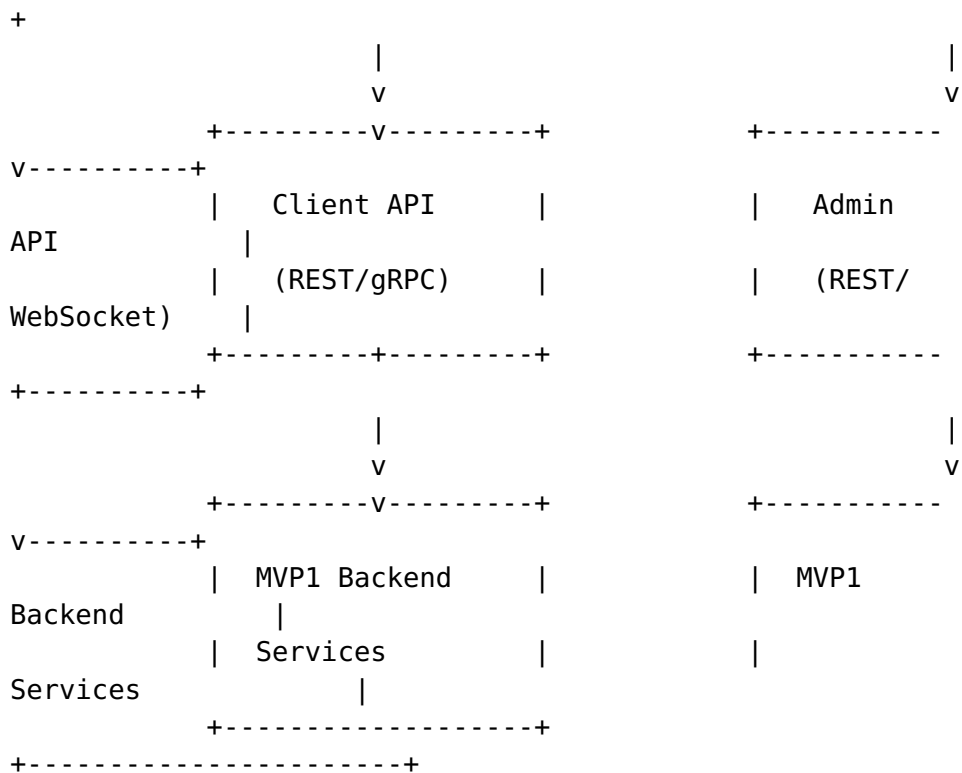
- helix-aurora for Aurora OS (Qt6/QML native application)
- helix-web for Chrome, Firefox, Edge, Safari (Browser Extension MV3 + PWA companion)

### Infrastructure:

- Unified CI/CD pipeline building all 8 platform artifacts from a single main branch
- Automated testing framework with property-based crypto testing and mock TUN integration tests
- Security audit pipeline with static analysis, dependency scanning, and fuzz testing
- OTA update system for all platforms

### 3.4 Relationship to Backend Services





All MVP2 client applications consume the same MVP1 backend services:

- **Client API:** Server list, latency data, connection credentials, account status
- **Authentication Service:** Login, token refresh, session management
- **Utils Service:** Feature flags, configuration, telemetry endpoints
- **Admin API:** Fleet management (enterprise), configuration deployment, analytics

## 4. Platform Coverage Matrix

### 4.1 Complete Platform Matrix

| Platform | OS      | OS Versions          | Arch          | UI Framework     | Rust Core | Build Target       |
|----------|---------|----------------------|---------------|------------------|-----------|--------------------|
| Desktop  | macOS   | 12+                  | x86_64, arm64 | Tauri v2 + React | Embedded  | .dylib (Universal) |
| Desktop  | Windows | 10/11                | x86_64, ARM64 | Tauri v2 + React | Embedded  | .dll .msi          |
| Desktop  | Linux   | 5.4+ (Ubuntu 22.04+, | x86_64, ARM64 | Tauri v2 + React | Embedded  | .so .AppImage      |

| Platform | OS        | OS Versions                                   | Arch          | UI Framework             | Rust Core              | Build Target      |
|----------|-----------|-----------------------------------------------|---------------|--------------------------|------------------------|-------------------|
|          |           | Fedora 39+)                                   |               |                          |                        |                   |
| Mobile   | Android   | API 26+ (8+)                                  | ARM64, x86_64 | Flutter 3.29+ + Impeller | .so via FRB            | .apk              |
| Mobile   | iOS       | 15+                                           | ARM64         | Flutter 3.29+ + Metal    | XCFramework via UniFFI | .ipa              |
| Mobile   | HarmonyOS | API 12+                                       | ARM64         | Flutter 3.22-ohos        | .so via FRB            | .hms              |
| Desktop  | Aurora OS | 4.x+                                          | ARM64         | Qt6 / QML                | .so via C FFI          | .rpm              |
| Web      | Browser   | Chrome 90+, Firefox 88+, Edge 90+, Safari 15+ | N/A           | Extension MV3 + PWA      | .wasm (crypto only)    | .zip (side store) |

## 4.2 Priority Tier Breakdown

| Tier               | Platforms                      | Priority | Business Rationale                                                                          |
|--------------------|--------------------------------|----------|---------------------------------------------------------------------------------------------|
| <b>TIER 1 (P0)</b> | macOS, Windows, Linux, Android | Highest  | Core revenue platforms representing >85% of addressable market                              |
| <b>TIER 2 (P1)</b> | iOS, HarmonyOS                 | High     | Strategic growth -- iOS for North American/European expansion, HarmonyOS for Chinese market |
| <b>TIER 3 (P2)</b> | Aurora OS                      | Medium   | Niche but strategically important for Russian market compliance                             |
| <b>TIER 4 (P3)</b> | Web Browser Extension          | Lower    | Companion feature for users who need browser-level protection without full-device VPN       |



### 4.3 Architecture Targets Per Platform

| Platform  | Native VPN API                            | Kill Switch Mechanism | Split Tunneling         | Biometric Auth     |
|-----------|-------------------------------------------|-----------------------|-------------------------|--------------------|
| macOS     | NetworkExtension (NEPacketTunnelProvider) | PF firewall anchor    | Per-app + route-based   | Touch ID           |
| Windows   | WFP + WinTUN driver                       | WFP sublayer rules    | Per-app + route-based   | Windows Hello      |
| Linux     | TUN/TAP + Netlink/D-Bus                   | nftables/iptables     | Route-based             | N/A                |
| Android   | VpnService + Builder                      | setBlocking(true)     | Per-app (package names) | Fingerprint        |
| iOS       | NEPacketTunnelProvider                    | NE includeRoutes      | Per-app (bundle IDs)    | Face ID / Touch ID |
| HarmonyOS | VpnExtensionAbility                       | Platform firewall     | Per-app                 | Fingerprint        |
| Aurora OS | ConnMan VPN (D-Bus)                       | ConnMan policy        | Route-based             | N/A                |
| Web       | chrome.proxy API                          | N/A (proxy scope)     | Domain-based            | N/A                |

## 5. Architecture at a Glance

### 5.1 The Rust Shared Core (helix-core)

The foundational architectural decision of MVP2 is the **Rust shared core** pattern: a single Rust library (helix-core) that contains all VPN protocol logic, cryptographic operations, connection management, and network orchestration. This core is compiled for each target platform and consumed through language-appropriate bindings.

**Workspace Structure:**

```
helix-core/
├─ Cargo.toml                # Workspace manifest
├─ crates/
│   └─ helix-core-api/      # Public API: all FFI
entry points
├─ | └─ Connection management
├─ | └─ Configuration parsing
├─ | └─ Event system (state changes, statistics)
└─ | └─ Account/Auth integration
```

```

|   |— helix-vpn-engine/           # Generic tunnel
management
|   |   |— Connection state machine
|   |   |— Routing logic
|   |   |— Firewall orchestration
|   |   |— DNS configuration
|   |— helix-wireguard/           # WireGuard protocol
implementation
|   |   |— Noise protocol handshake
|   |   |— ChaCha20-Poly1305 packet crypto
|   |   |— Timer management (keepalive, rekey)
|   |   |— Packet encapsulation/decapsulation
|   |— helix-crypto/              # Cryptographic primitives
|   |   |— X25519 key exchange
|   |   |— HKDF key derivation
|   |   |— BLAKE2s hashing
|   |   |— Post-quantum ML-KEM (optional)
|   |— helix-network/             # HTTP/API client
|   |   |— REST API client
|   |   |— Latency testing
|   |   |— gRPC support
|   |— helix-platform-abstraction/ # OS adapter trait
implementations
|   |   |— TUN device creation
|   |   |— Route management
|   |   |— DNS configuration
|   |   |— Kill switch firewall rules
|   |   |— Secure key storage
|— bindings/
|   |— uniffi/                    # UniFFI UDL + generated
Kotlin/Swift
|   |— wasm/                      # wasm-bindgen target
config
|— tests/
|   |— integration/               # Cross-platform
integration tests
|   |— fuzz/                      # Crypto fuzzing targets

```

## 5.2 UI Layer Separation Per Platform

Each platform implements **only** a thin UI layer that communicates with the shared core through a platform-specific binding:

| Platform Group            | UI Framework | Binding Mechanism            | UI Code Type       |
|---------------------------|--------------|------------------------------|--------------------|
| Desktop (macOS/Win/Linux) | Tauri v2     | Tauri Commands (IPC)         | React + TypeScript |
|                           | Flutter      | flutter_rust_bridge + UniFFI | Dart (Material 3   |

| Platform Group                        | UI Framework                | Binding Mechanism          | UI Code Type                   |
|---------------------------------------|-----------------------------|----------------------------|--------------------------------|
| Mobile<br>(Android/iOS/<br>HarmonyOS) |                             |                            | + Cupertino Adaptive)          |
| Aurora OS                             | Qt6 / QML                   | Direct C FFI<br>(cbindgen) | QML + C++                      |
| Web                                   | Browser<br>Extension<br>MV3 | wasm-bindgen               | JavaScript /<br>React          |
| Admin                                 | Tauri v2                    | Tauri Commands<br>(IPC)    | React +<br>Admin<br>Components |

**Critical principle:** UI layers contain **zero VPN protocol logic**. They are responsible for:

- Rendering the user interface
- Handling user input
- Displaying connection state and statistics
- Managing local settings/preferences
- Delegating all VPN operations to the Rust core

## 5.3 Platform Adapter Pattern

Each platform implements a **Platform Adapter** that bridges the Rust core to native OS VPN APIs. Adapters handle:

- **TUN virtual interface creation** -- Creating and configuring the virtual network interface
- **Routing table manipulation** -- Adding/removing routes, setting interface metrics
- **DNS server configuration** -- Per-OS DNS configuration
- **Firewall rules** -- Kill switch implementation via platform-specific firewall APIs
- **Socket protection** -- Marking transport sockets to bypass the VPN tunnel
- **Network change monitoring** -- Detecting WiFi/cellular transitions for auto-reconnect
- **Secure key storage** -- Integration with OS keychain/keystore

The adapter implements the PlatformAdapter trait defined in helix-platform-abstraction, ensuring consistent behavior across all platforms while allowing each OS to use its native APIs.

## 5.4 Code Reuse Analysis

SHARED RUST CORE (~78% of total codebase)

└─ helix-wireguard ..... 8,000 LOC -- 100% shared (all

platforms)

- └─ helix-vpn-engine ..... 6,000 LOC -- 100% shared
- └─ helix-crypto ..... 2,000 LOC -- 100% shared
- └─ helix-network ..... 3,000 LOC -- 100% shared
- └─ helix-core-api ..... 4,000 LOC -- 100% shared

PLATFORM-SPECIFIC CODE (~22% of total codebase)

- └─ Desktop-Specific ..... ~8% (Tauri commands + 3 OS adapters)
- └─ Mobile-Specific ..... ~10% (Flutter UI + 3 OS adapters)
- └─ Niche Platforms ..... ~4% (Qt UI + ConnMan adapter, Web extension)

**Effective Reuse Percentages by Platform:**

| Platform  | Rust Core Reuse | Platform-Specific            | Binding Layer     | Total Reuse |
|-----------|-----------------|------------------------------|-------------------|-------------|
| macOS     | 85%             | 10% (NetworkExtension, PF)   | 5% (Tauri IPC)    | 85%         |
| Windows   | 80%             | 15% (WFP, WinTUN)            | 5% (Tauri IPC)    | 80%         |
| Linux     | 85%             | 10% (netlink, nftables)      | 5% (Tauri IPC)    | 85%         |
| Android   | 72%             | 20% (VpnService)             | 8% (FRB)          | 72%         |
| iOS       | 72%             | 20% (NEPacketTunnelProvider) | 8% (FRB + UniFFI) | 72%         |
| HarmonyOS | 70%             | 20% (VpnExtensionAbility)    | 10% (FRB)         | 70%         |
| Aurora OS | 75%             | 15% (ConnMan D-Bus)          | 10% (C FFI)       | 75%         |
| Web       | 45%             | 40% (proxy-based, no TUN)    | 15% (JS glue)     | 45%         |

## 6. Technology Stack Summary

### 6.1 Core Layer -- Rust

| Component         | Technology                           | Purpose                                  |
|-------------------|--------------------------------------|------------------------------------------|
| Language          | Rust 1.78+                           | Memory-safe systems programming          |
| Async Runtime     | tokio                                | Multi-threaded async I/O                 |
| WireGuard         | boringtun (Cloudflare)               | Protocol implementation                  |
| TLS Stack         | rustls                               | TLS 1.2/1.3 with ring crypto             |
| Crypto Primitives | ring, x25519-dalek, chacha20poly1305 | X25519, ChaCha20-Poly1305, HKDF, BLAKE2s |
| Post-Quantum      | aws-lc-rs (ML-KEM-768)               | Hybrid X25519 + ML-KEM key exchange      |
| TUN Interface     | tun-rs                               | Cross-platform virtual interface         |
| DNS               | hickory-resolver                     | DoH/DoT/DoQ DNS resolution               |
| QUIC/MASQUE       | quinn + h3                           | HTTP/3 and CONNECT-UDP proxying          |
| Serialization     | serde + serde_json                   | Configuration and API data               |
| Logging           | tracing + tracing-subscriber         | Structured logging                       |
| Memory Safety     | zeroize + secrecy                    | Secure key destruction                   |

### 6.2 Desktop Layer -- Tauri v2

| Component        | Technology                                           | Purpose                  |
|------------------|------------------------------------------------------|--------------------------|
| Framework        | Tauri v2                                             | App shell, IPC, bundling |
| Frontend         | React 19 + TypeScript                                | UI components            |
| State Management | Zustand / Valtio                                     | Application state        |
| WebView          | WKWebView (macOS), WebView2 (Win), WebKitGTK (Linux) | Native rendering         |

| Component | Technology                                                     | Purpose                           |
|-----------|----------------------------------------------------------------|-----------------------------------|
| IPC       | Tauri Commands                                                 | Capability-based Rust-JS bridge   |
| Bundling  | Tauri CLI                                                      | .dmg, .msi, .AppImage, .deb, .rpm |
| Plugins   | tauri-plugin-notification, autostart, updater, single-instance | Native integrations               |

## 6.3 Mobile Layer -- Flutter

| Component        | Technology                              | Purpose                         |
|------------------|-----------------------------------------|---------------------------------|
| Framework        | Flutter 3.29+                           | Cross-platform UI               |
| Renderer         | Impeller (iOS default, Android default) | GPU-accelerated 60 FPS          |
| Language         | Dart 3+                                 | UI + business logic             |
| State Management | flutter_bloc / Riverpod                 | Reactive state                  |
| Rust Bridge      | flutter_rust_bridge v2.0+               | Dart-Rust FFI generation        |
| iOS Bindings     | UniFFI (Mozilla)                        | Swift-Kotlin binding generation |
| HTTP Client      | dio                                     | Dart HTTP client                |
| Storage          | hive                                    | Local key-value storage         |
| HarmonyOS        | Flutter 3.22-ohos (community)           | ohos embedding layer            |

## 6.4 Aurora OS Layer -- Qt6/QML

| Component   | Technology                | Purpose                                |
|-------------|---------------------------|----------------------------------------|
| Framework   | Qt 6.7+                   | Native Aurora OS framework             |
| UI Language | QML                       | Declarative UI (Sailfish Silica-style) |
| Backend     | C++                       | QObject controllers, D-Bus integration |
| Rust Bridge | Direct C FFI via cbindgen | C header generation                    |
| Network     | Qt D-Bus                  | ConnMan VPN integration                |

## 6.5 Web Layer -- Browser Extension

| Component                | Technology                      | Purpose                              |
|--------------------------|---------------------------------|--------------------------------------|
| <b>Extension API</b>     | Manifest V3                     | Chrome/Firefox/Edge/Safari extension |
| <b>Rust Usage</b>        | WASM (wasm-bindgen + wasm-pack) | Cryptographic operations only        |
| <b>Proxy</b>             | chrome.proxy / browser.proxy    | PAC script configuration             |
| <b>Native Bridge</b>     | Native Messaging API            | Communication with helix-desktop     |
| <b>PWA</b>               | Service Worker + Web APIs       | Standalone web companion             |
| <b>Crypto in Browser</b> | WASM-compiled from helix-crypto | Key generation, encryption           |

## 7. Key Features

### 7.1 Multi-Protocol Support

MVP2 clients support multiple VPN protocols, selectable per connection:

| Protocol           | Status    | Description                                                                       |
|--------------------|-----------|-----------------------------------------------------------------------------------|
| <b>WireGuard</b>   | Primary   | Default protocol -- Noise IK handshake, ChaCha20-Poly1305, minimal attack surface |
| <b>Shadowsocks</b> | Secondary | SIP022 AEAD-2022 -- obfuscation for censorship circumvention                      |
| <b>MASQUE</b>      | Secondary | RFC 9298 CONNECT-UDP over HTTP/3 -- deep packet inspection resistance             |
| <b>Multi-Hop</b>   | Advanced  | Entry-exit chaining through two WireGuard tunnels for enhanced anonymity          |

Protocol selection is driven by the **auto-detection engine**, which tests available protocols against the target server and selects the optimal option based on latency, throughput, and network conditions.

## 7.2 Kill Switch

The kill switch is a **fail-closed security mechanism** that blocks all network traffic when the VPN connection drops unexpectedly. Platform implementations:

- **macOS:** PF (Packet Filter) firewall anchor -- blocks all outbound except VPN tunnel interface (utun\*)
- **Windows:** WFP (Windows Filtering Platform) sublayer -- blocks all outbound, allows only VPN interface
- **Linux:** nftables/iptables -- sets output policy to DROP, adds accept rule for tun0
- **Android:** VpnService.setBlocking(true) -- blocks at the OS VPN layer
- **iOS:** NE includeRoutes with no default route -- routes only through tunnel
- **HarmonyOS:** Platform firewall rules via VpnExtensionAbility
- **Aurora OS:** ConnMan VPN policy configuration
- **Web:** N/A (browser extension operates in proxy scope only)

**Target response time:** < 100ms from connection drop detection to full traffic block.

## 7.3 Split Tunneling

Split tunneling allows selective routing of traffic through or around the VPN tunnel:

- **Per-App Split Tunneling** (Android, iOS, macOS, Windows): Route specific applications through VPN while others bypass
- **Route-Based Split Tunneling** (all platforms): Route specific IP ranges/CIDR blocks through VPN
- **Inverse Split Tunneling:** Route everything through VPN except specified apps/ranges
- **Dynamic Rules:** Rule evaluation happens in the shared Rust core; enforcement via platform routing APIs

## 7.4 Auto-Connect on Untrusted Networks

The client monitors network connectivity and automatically establishes a VPN connection when:

- Connecting to an open/public WiFi network (no encryption)
- Connecting to a network not on the trusted network list
- Device boots or wakes from sleep
- Network interface changes (WiFi to cellular, etc.)

This feature uses the **Network Trust Framework**, which maintains a database of known-safe networks (home, office) and triggers auto-connect for all others.



## 7.5 Biometric Authentication

Mobile platforms support biometric authentication for app unlock and connection authorization:

| Platform  | Biometric Method          | API                             |
|-----------|---------------------------|---------------------------------|
| iOS       | Face ID / Touch ID        | LocalAuthentication framework   |
| Android   | Fingerprint / Face Unlock | BiometricPrompt API             |
| HarmonyOS | Fingerprint               | HMS FIDO API                    |
| macOS     | Touch ID                  | LocalAuthentication             |
| Windows   | Windows Hello             | Windows.Security.Credentials.UI |

## 7.6 Obfuscation and Anti-Censorship

Traffic obfuscation techniques to evade deep packet inspection and network-level censorship:

| Technique              | Description                                                                           | Use Case                                     |
|------------------------|---------------------------------------------------------------------------------------|----------------------------------------------|
| <b>DAITA</b>           | Defense Against AI-guided Traffic Analysis -- packet padding and timing randomization | Advanced traffic analysis resistance         |
| <b>Shadowsocks</b>     | SIP022 AEAD-2022 encrypted proxy transport                                            | Censorship circumvention (China, Iran, etc.) |
| <b>Fragmentation</b>   | IP-level packet fragmentation to evade signature-based detection                      | Bypassing simple DPI filters                 |
| <b>Domain Fronting</b> | Routing traffic through CDN domains                                                   | Blocking evasion                             |

## 7.7 Multi-Hop Connections

Multi-hop chains traffic through two VPN servers sequentially:

Client --> [Entry Node: WireGuard] --> [Exit Node: WireGuard] --> Internet

- Entry node sees client IP but not destination
- Exit node sees destination but not client IP
- Each hop uses independent WireGuard keys and sessions
- Latency penalty: typically 15-30% increase over single-hop

- Available on all platforms except Web (which lacks TUN access)

## 7.8 Enterprise & Production Readiness Additions (Revision 2)

The initial MVP2 draft focused heavily on consumer-facing VPN mechanics (protocols, kill switch, split tunneling). This revision closes the enterprise-operations and production-hardening gaps identified in the cross-document gap analysis. Each row is detailed in the cited platform document's own "Enterprise Hardening" / "Production Readiness" section.

| Concern                                  | Summary                                                                                                                                                                                                                                                                                                                                                                                                              | Primary Document                                                                               |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Multi-Account / Profile Switching</b> | A single app install supports switching between multiple Helix accounts (personal + team/family plan), with isolated per-profile settings, keys, and connection history                                                                                                                                                                                                                                              | MVP2_UI_UX_SPEC.md \$UX flows, MVP2_DESKTOP_APPS.md / MVP2_MOBILE_APPS.md Enterprise Hardening |
| <b>Enterprise SSO</b>                    | Organization-configured SAML/OIDC login via system browser (ASWebAuthenticationSession/Custom Tabs on mobile, system browser on desktop), bridging to the MVP1 Authentication Service's existing OAuth2/OIDC support                                                                                                                                                                                                 | MVP2_SECURITY_PERFORMANCE.md, per-platform Enterprise Hardening sections                       |
| <b>MDM &amp; Centralized Policy Push</b> | Bulk configuration and policy enforcement (forced kill switch, allowed protocols, DNS policy) pushed from the helix-admin dashboard via the MVP1 Admin API to enrolled devices — macOS/Windows Configuration Profiles + Group Policy/Intune, Android Enterprise managed configuration, Apple MDM managed app config, Chrome/Firefox enterprise extension policies<br>Subscription/plan-tier verification against the | MVP2_SECURITY_PERFORMANCE.md §10 (new), per-platform Enterprise Hardening sections             |

| Concern                                                | Summary                                                                                                                                                                                                                      | Primary Document                                                         |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>License / Entitlement Checks</b>                    | Client API with a signed, tamper-evident offline-grace cache so connectivity briefly surviving an API outage does not silently disable protection                                                                            | MVP2_SECURITY_PERFORMANCE.md, per-platform Enterprise Hardening sections |
| <b>Crash Reporting &amp; Telemetry (Privacy-First)</b> | Opt-in, privacy-scrubbed crash reporting (Sentry or equivalent) and aggregated, anonymized telemetry gated behind an explicit first-run consent screen with granular toggles, feeding the MVP1 Utils Service                 | MVP2_SECURITY_PERFORMANCE.md §10 (new)                                   |
| <b>Update Rollback &amp; Staged/Canary Rollout</b>     | Every platform's auto-update mechanism supports a canary ring (internal → 1% → 10% → 50% → 100%) with automatic rollback on crash-rate/error-rate regression, coordinated through the MVP1 Utils Service feature-flag system | MVP2_SECURITY_PERFORMANCE.md §10 (new), MVP2_IMPLEMENTATION_ROADMAP.md   |
| <b>Supply-Chain Security</b>                           | Reproducible Rust core builds (cargo --locked, pinned toolchain), per-artifact SBOM generation (cargo-cyclonedx/syft), and signed release manifests across all 14+ build artifacts                                           | MVP2_SHARED_CORE.md §5.3 (new), MVP2_SECURITY_PERFORMANCE.md §10 (new)   |
| <b>Accessibility &amp; Localization</b>                | WCAG 2.1 AA target across all platform UI layers (screen reader bridges for Tauri/Flutter/QML non-native widget trees), full i18n string externalization, and RTL layout support                                             | MVP2_UI_UX_SPEC.md (expanded)                                            |
| <b>Offline / Degraded-Network Behavior</b>             | Defined reconnect/backoff strategy, cached-server-list TTL, and explicit UI states for no-network and degraded-network conditions rather than                                                                                | MVP2_UI_UX_SPEC.md, per-platform Enterprise Hardening sections           |

| Concern                   | Summary                                                                                                                                                                                                      | Primary Document      |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| Design-System Consistency | treating them as unhandled edge cases                                                                                                                                                                        |                       |
|                           | The OpenDesign token system (docs/design/README.md) — light/dark themes, brand colors, typography, spacing, component tokens — is the mandatory basis for every platform's UI, not an aspirational reference | MVP2_UI_UX_SPEC.md §1 |

## 8. Documentation Structure

### 8.1 MVP2 Document Registry

The following documents comprise the complete MVP2 specification suite. **Revision 2 correction:** the table below previously referenced ten never-created filenames (MVP2\_DESKTOP.md, MVP2\_MOBILE.md, MVP2\_AURORA.md, MVP2\_WEB.md, MVP2\_SECURITY.md, MVP2\_DEVOPS.md, MVP2\_QA.md) all marked "Planned" — these were the working titles from the original plan.md Stage 3 brief (16 documents planned). During synthesis, DevOps/CI-CD, QA/Testing, and Security/Performance content was consolidated into fewer, denser documents rather than split as originally scoped; the table now reflects the actual delivered filenames and their **Final Draft** status.

| # | Document Name                                   | File Path            | Description                                                                                                     | Target Audience                          |
|---|-------------------------------------------------|----------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------|
| 1 | <b>MVP2 Overview</b><br>(this document)         | MVP2_OVERVIEW.md     | Executive summary, platform matrix, architecture overview, success criteria, risk summary                       | Executive, Engineers, Leads              |
| 2 | <b>MVP2 Architecture &amp; Technology Stack</b> | MVP2_ARCHITECTURE.md | Detailed architecture decisions, technology justifications, code reuse analysis, data flow diagrams, connection | Engineers, Leads, Platform Teams, DevOps |

| # | Document Name                         | File Path             | Description                                                                                                                                                                                                                                         | Target Audience                                       |
|---|---------------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 3 | <b>MVP2 Shared Core Specification</b> | MVP2_SHARED_CORE.md   | <p>lifecycle state machine, enterprise hardening overview</p> <p>helix-core API design, module architecture, protocol implementations, FFI bindings, testing strategy, security architecture, performance budgets, supply-chain/build-integrity</p> | Rust Core Team, Platform Engineering, Security Review |
| 4 | <b>MVP2 Desktop Specification</b>     | MVP2_DESKTOP_APPS.md  | <p>Tauri v2 application spec (macOS/Windows/Linux), IPC commands, UI components, code-signing/notarization, MDM/enterprise deployment, auto-update + rollback</p>                                                                                   | Desktop Team, Engineering                             |
| 5 | <b>MVP2 Mobile Specification</b>      | MVP2_MOBILE_APPS.md   | <p>Flutter application spec (Android/iOS/HarmonyOS), FRB integration, platform channel design, mobile UI patterns, app-store review, MDM/managed config, background execution limits</p>                                                            | Mobile Team, Engineering                              |
| 6 | <b>MVP2 Aurora OS Specification</b>   | MVP2_AURORA_CLIENT.md | <p>Qt6/QML application spec, C FFI bridge, ConnMan integration,</p>                                                                                                                                                                                 | Aurora Platform Team                                  |

| #  | Document Name                           | File Path                      | Description                                                                                                                                                                                     | Target Audience                           |
|----|-----------------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| 7  | <b>MVP2 Web Extension Specification</b> | MVP2_WEB_CLIENT.md             | Sailfish Silica UI patterns, Aurora Store submission<br>Browser extension spec, WASM crypto, proxy modes, native messaging, PWA companion, store review, enterprise policy push                 | Web T                                     |
| 8  | <b>MVP2 Security &amp; Performance</b>  | MVP2_SECURITY_PERFORMANCE.md   | Threat model, kill switch, leak prevention, obfuscation, performance budgets, CI/CD build pipeline, supply-chain security, staged/canary rollout, compliance checklists (SOC2, ISO 27001, GDPR) | Secur<br>Team,<br>DevO,<br>Comp<br>Office |
| 9  | <b>MVP2 UI/UX Design Specification</b>  | MVP2_UI_UX_SPEC.md             | OpenDesign-token-based design system, component library, screen specifications, accessibility, localization/i18n                                                                                | UI/UX<br>Design<br>Platform<br>Teams      |
| 10 | <b>MVP2 Implementation Roadmap</b>      | MVP2_IMPLEMENTATION_ROADMAP.md | 36-week phased plan (9 phases) with milestones, acceptance criteria, risk-adjusted timeline, team structure                                                                                     | Engin<br>Leads                            |

Note: the original plan.md Stage 3 brief additionally scoped standalone MVP2\_TECHNOLOGY\_STACK.md, MVP2\_BUILD\_SYSTEM.md, MVP2\_TESTING\_QA.md, MVP2\_API\_REFERENCE.md, and MVP2\_DEVELOPMENT\_GUIDE.md documents. Their content was

folded into documents 2, 3, 8, and 10 above rather than maintained as separate files — this is a deliberate consolidation, not a scope gap. See `plan.md`'s Revision 2 note for the full reconciliation.

## 8.2 Document Reading Guide

### For Executive Stakeholders:

- Read: Document 1 (this overview) -- provides all high-level information needed for decision-making
- Reference: Document 2 for architecture details if needed

### For Engineering Managers:

- Read: Documents 1, 2, 3 -- complete understanding of architecture and core API
- Reference: Documents 4-7 for platform-specific implementation details

### For Platform Engineers:

- Read: Documents 1, 2, 3 -- understand the shared core and binding mechanisms
- Read: Your platform-specific document (4, 5, 6, or 7)
- Reference: Document 9 for build and deployment procedures

### For Security Review:

- Read: Documents 1, 2, 3 -- architecture and core implementation
- Read: Document 8 -- security-specific requirements and compliance

### For QA Engineers:

- Read: Document 1 -- understand scope and success criteria
- Read: Document 10 -- detailed test plans and acceptance criteria

---

## 9. Success Criteria

### 9.1 Functional Criteria

| ID    | Criterion                         | Target                          | Measurement Method              |
|-------|-----------------------------------|---------------------------------|---------------------------------|
| SC-01 | All 8 platform targets functional | 8/8 platforms connect, transfer | Automated end-to-end test suite |

| ID    | Criterion                          | Target                                                      | Measurement Method                     |
|-------|------------------------------------|-------------------------------------------------------------|----------------------------------------|
|       |                                    | data, disconnect correctly                                  |                                        |
| SC-02 | WireGuard protocol stable          | 100% handshake success rate under normal network conditions | 1,000 connection stress test           |
| SC-03 | Shadowsocks obfuscation functional | Successful connection through simulated DPI/firewall        | Lab environment with Suricata/Zeek DPI |
| SC-04 | MASQUE transport functional        | CONNECT-UDP tunnel established via HTTP/3                   | Server-side validation                 |
| SC-05 | Multi-hop chaining functional      | Traffic routes through entry + exit nodes                   | Server log correlation                 |
| SC-06 | Kill switch operates correctly     | Zero packet leakage during unexpected disconnect            | tcpdump/wireshark capture analysis     |
| SC-07 | Split tunneling routes correctly   | App/route traffic split verified per rules                  | Network monitoring on test device      |
| SC-08 | Auto-connect triggers correctly    | Connection established within 5s of untrusted network join  | Automated network transition tests     |
| SC-09 | Biometric auth functions correctly | 100% unlock success with enrolled biometric                 | Manual testing per platform            |
| SC-10 | OTA updates install correctly      | Update downloads and installs without user intervention     | CI/CD simulation                       |

## 9.2 Performance Criteria

| ID    | Criterion  | Desktop Target              | Mobile Target                                     | Measurement Method         |
|-------|------------|-----------------------------|---------------------------------------------------|----------------------------|
| SC-11 | Code reuse | >= 70% across all platforms | >= 70% across all platforms (same cross-platform) | Source code analysis (LOC) |



| ID    | Criterion               | Desktop Target      | Mobile Target                              | Measurement Method                  |
|-------|-------------------------|---------------------|--------------------------------------------|-------------------------------------|
|       |                         |                     | target, not desktop/mobile-differentiated) |                                     |
| SC-12 | Bundle size             | < 15 MB compressed  | < 25 MB compressed                         | ls -la of release artifact          |
| SC-13 | Connection time         | < 1 second          | < 2 seconds                                | WireGuard handshake duration timer  |
| SC-14 | Kill switch response    | < 100 ms            | < 200 ms                                   | tcpdump + kernel timestamp analysis |
| SC-15 | Throughput (WireGuard)  | >= 500 Mbps         | >= 100 Mbps                                | iperf3 over tunnel                  |
| SC-16 | CPU usage (idle)        | < 1%                | < 2%                                       | OS process monitor                  |
| SC-17 | CPU usage (loaded)      | < 25% at 500 Mbps   | < 40% at 100 Mbps                          | OS process monitor                  |
| SC-18 | Memory footprint        | < 80 MB             | < 120 MB                                   | RSS measurement (ps)                |
| SC-19 | Battery impact (hourly) | N/A                 | < 5%                                       | OS battery monitoring               |
| SC-20 | UI responsiveness       | < 100 ms UI updates | < 100 ms UI updates                        | Frame timing / Flutter devtools     |

### 9.3 Security Criteria

| ID    | Criterion                              | Target                                                         |
|-------|----------------------------------------|----------------------------------------------------------------|
| SC-21 | Zero critical security vulnerabilities | 0 critical/high CVEs in dependencies at release                |
| SC-22 | Zero memory safety issues              | 0 Miri warnings, 0 ASan/MSan detections                        |
| SC-23 | No DNS leaks                           | 100% pass on dnsleaktest.com and ipleak.net                    |
| SC-24 | No IPv6 leaks                          | 100% pass on IPv6 leak tests                                   |
| SC-25 | Key material protection                | All private keys stored in OS secure storage; zeroized on drop |
| SC-26 | Crypto audit readiness                 |                                                                |

| ID | Criterion | Target                                                              |
|----|-----------|---------------------------------------------------------------------|
|    |           | All cryptographic code passes independent audit (planned post-MVP2) |

### 9.4 Quality Criteria

| ID    | Criterion                              | Target                                 |
|-------|----------------------------------------|----------------------------------------|
| SC-27 | Unit test coverage (core)              | >= 85% line coverage                   |
| SC-28 | Unit test coverage (platform adapters) | >= 70% line coverage                   |
| SC-29 | Integration test pass rate             | 100% on all supported platforms        |
| SC-30 | Crash-free rate                        | >= 99.9% over 30-day monitoring period |

## 10. Risk Summary

### 10.1 Top 10 Risks

| Rank | Risk                                      | Probability | Impact | Risk Score    | Mitigation Strategy                                                                                                                                                              |
|------|-------------------------------------------|-------------|--------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>Flutter HarmonyOS breaking changes</b> | Medium      | High   | <b>High</b>   | Pin to specific ohos embedding version (v3.22.0-ohos); maintain fork if upstream breaks; allocate 2-week buffer for adapter rework                                               |
| 2    | <b>Tauri v2 mobile immaturity</b>         | Medium      | Medium | <b>Medium</b> | Desktop is production-stable; for mobile-adjacent features, write native plugins in Swift/Kotlin; track Tauri mobile roadmap closely                                             |
| 3    | <b>iOS App Store rejection</b>            | Low         | High   | <b>Medium</b> | Prepare comprehensive privacy policy; document legitimate VPN use cases; comply with all NEPacketTunnelProvider entitlement requirements; engage Apple developer relations early |
| 4    |                                           | High        | Medium | <b>Medium</b> |                                                                                                                                                                                  |

| Rank | Risk                                                 | Probability | Impact | Risk Score    | Mitigation Strategy                                                                                                                                                         |
|------|------------------------------------------------------|-------------|--------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      | <b>Android OEM battery optimization interference</b> |             |        |               | Implement foreground service with persistent notification; use WorkManager for background tasks; provide battery exemption guides in onboarding; test on top 20 OEM devices |
| 5    | <b>Rust core binary size exceeds targets</b>         | Medium      | Medium | <b>Medium</b> | Apply all size optimizations (LTO, panic=abort, strip); use ring over OpenSSL; profile per-feature binary contribution; consider feature flagging for niche protocols       |
| 6    | <b>WFP rule conflicts on Windows</b>                 | Medium      | Medium | <b>Medium</b> | Implement careful layer weight positioning in WFP sublayer; add rule restoration on BFE (Base Filtering Engine) restart alongside third-party antivirus/firewall software   |
| 7    | <b>UniFFI async limitations</b>                      | Medium      | Medium | <b>Medium</b> | Use Tokio Runtime pattern with LazyLock<Runtime>; implement sync FFI with callbacks as fallback; test async behavior thoroughly on iOS                                      |
| 8    | <b>Cross-compilation toolchain fragility</b>         | Medium      | Low    | <b>Low</b>    | Pin all toolchain versions; use cross-rs Docker images; maintain build matrix in CI; document all linker flags per target                                                   |
| 9    | <b>Windows driver signing (WinTUN)</b>               | Low         | Medium | <b>Low</b>    | WinTUN is already signed by WireGuard Foundation; no custom driver needed; monitor for signing cert expiration                                                              |
| 10   | <b>Web extension</b>                                 | Low         | Medium | <b>Low</b>    | Design for Manifest V3 compliance; avoid proprietary APIs that stores restrictions                                                                                          |

| Rank | Risk                 | Probability | Impact | Risk Score | Mitigation Strategy                                                                  |
|------|----------------------|-------------|--------|------------|--------------------------------------------------------------------------------------|
|      | store policy changes |             |        |            | provide sideload installation as fallback<br>monitor Chrome Web Store policy updates |

### 10.2 Platform-Specific Risks

| Platform  | Specific Risk                                                                   | Mitigation                                                                            |
|-----------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| macOS     | NetworkExtension entitlements require Apple Developer account and manual review | Apply early; include in CI with developer cert                                        |
| Windows   | WFP complexity; antivirus false positives on VPN binaries                       | Code signing cert (Extended Validation); Microsoft SmartScreen reputation building    |
| Linux     | Distribution diversity (glibc versions, init systems, DEs)                      | Target Ubuntu 22.04 LTS and Fedora 39+ as primary; AppImage for broad compatibility   |
| Android   | Background execution limits (Doze, App Standby); OEM-specific behavior          | Foreground service + wake locks; whitelist guides for popular OEMs                    |
| iOS       | NEPacketTunnelProvider memory limits (~15 MB); strict App Store review          | Memory-optimize core; use native Network Extension patterns; pre-submission review    |
| HarmonyOS | Community Flutter embedding maturity; limited documentation                     | Active community engagement; maintain fork with patches; fallback to native if needed |
| Aurora OS |                                                                                 | Direct engagement with                                                                |

| Platform | Specific Risk                                        | Mitigation                                                                     |
|----------|------------------------------------------------------|--------------------------------------------------------------------------------|
| Web      | Limited developer documentation; niche hardware      | Aurora OS team; test on reference hardware early                               |
|          | No raw socket/TUN access; limited to proxy mode only | Design three operating modes (proxy, native bridge, P2P); graceful degradation |

### 10.3 Technical Risks

| Risk                                            | Description                                                       | Mitigation                                                                                                                             |
|-------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Rust FFI complexity</b>                      | Managing memory across language boundaries is error-prone         | Use UniFFI and flutter_rust_bridge for automated binding generation; property-based testing for FFI boundaries; Miri for memory safety |
| <b>Protocol interoperability</b>                | WireGuard/Shadowsocks/MASQUE may have edge-case incompatibilities | Extensive integration testing with reference server implementations; protocol compliance test suite                                    |
| <b>Performance regression</b>                   | New platform features may degrade core performance                | Continuous benchmarking in CI; flamegraph profiling; performance budgets per module                                                    |
| <b>Security vulnerabilities in dependencies</b> | Third-party crates may have undiscovered vulnerabilities          | cargo audit in CI; Dependabot alerts; minimal dependency tree; vendoring critical crates                                               |

### 10.4 Timeline Risks

| Risk                           | Impact                    | Mitigation                                                                                                                                 |
|--------------------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Core development delays</b> | All platform work blocked | Front-load core development; define clear API contract; allow parallel UI mockup work<br>Tiered delivery (P0 first); maintain per-platform |

| <b>Risk</b>                       | <b>Impact</b>                             | <b>Mitigation</b>                                                                                    |
|-----------------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Platform-specific blockers</b> | Individual platform delayed               | buffers; allow scope reduction for P2/P3                                                             |
| <b>Integration complexity</b>     | Final integration phase exceeds estimates | Continuous integration from week 1; automated cross-platform testing; staged integration checkpoints |

## 11. Glossary

### 11.1 Key Terms and Abbreviations

| <b>Term</b>            | <b>Definition</b>                                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AEAD</b>            | Authenticated Encryption with Associated Data -- encryption mode that provides both confidentiality and integrity (e.g., ChaCha20-Poly1305, AES-GCM) |
| <b>BFE</b>             | Base Filtering Engine -- Windows kernel-mode service that manages WFP firewall rules                                                                 |
| <b>BoringTun</b>       | Cloudflare's open-source Rust implementation of the WireGuard protocol                                                                               |
| <b>CBindgen</b>        | Tool that generates C/C++ headers from Rust code for FFI integration                                                                                 |
| <b>DAITA</b>           | Defense Against AI-guided Traffic Analysis -- Mullvad's traffic obfuscation technique using padding and timing randomization                         |
| <b>DPI</b>             | Deep Packet Inspection -- network traffic analysis technique used by ISPs and governments to detect and block VPN traffic                            |
| <b>DoH / DoT / DoQ</b> | DNS-over-HTTPS / DNS-over-TLS / DNS-over-QUIC -- encrypted DNS resolution protocols                                                                  |
| <b>FFI</b>             | Foreign Function Interface -- mechanism for code written in one language to call code written in another language                                    |

| Term                          | Definition                                                                                               |
|-------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>FRB</b>                    | flutter_rust_bridge -- code generation tool for Dart-Rust FFI integration                                |
| <b>Impeller</b>               | Flutter's rendering engine, replacing Skia for GPU-accelerated, jank-free performance                    |
| <b>JNI</b>                    | Java Native Interface -- mechanism for Java/Kotlin code to call native code (C/C++/Rust) on Android      |
| <b>Kill Switch</b>            | Security feature that blocks all network traffic when the VPN connection drops, preventing IP/DNS leaks  |
| <b>KEM</b>                    | Key Encapsulation Mechanism -- post-quantum cryptographic primitive for secure key exchange              |
| <b>MASQUE</b>                 | Multiplexed Application Substrate over QUIC Encryption -- IETF protocol for proxying UDP/TCP over HTTP/3 |
| <b>ML-KEM</b>                 | Module-Lattice-based Key Encapsulation Mechanism -- NIST post-quantum cryptography standard              |
| <b>MVP</b>                    | Minimum Viable Product -- a product phase with just enough features to be usable by early customers      |
| <b>NEPacketTunnelProvider</b> | iOS/macOS Network Extension class for implementing custom VPN protocols at the packet level              |
| <b>Noise Protocol</b>         | Cryptographic framework for building lightweight protocols, used as the foundation of WireGuard          |
| <b>PAC</b>                    | Proxy Auto-Configuration -- JavaScript-based proxy configuration used by browser extensions              |
| <b>PF</b>                     | Packet Filter -- BSD/macOS firewall framework used for kill switch implementation                        |
| <b>PWA</b>                    | Progressive Web App -- web application that can be installed and run like a native app                   |
| <b>SIP022</b>                 |                                                                                                          |

| <b>Term</b>            | <b>Definition</b>                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------|
|                        | Shadowsocks Improved Proposal 022 -- defines the AEAD-2022 cipher specification for Shadowsocks      |
| <b>Split Tunneling</b> | Routing only selected traffic through the VPN while allowing other traffic direct internet access    |
| <b>Tauri</b>           | Rust-based framework for building lightweight desktop applications using web frontend technologies   |
| <b>TUN</b>             | Virtual network interface that operates at Layer 3 (IP), used to intercept and inject IP packets     |
| <b>UniFFI</b>          | Mozilla's multi-language binding generator for Rust (generates Kotlin, Swift, Python bindings)       |
| <b>VpnService</b>      | Android API for creating custom VPN connections via a TUN interface                                  |
| <b>WASM</b>            | WebAssembly -- binary instruction format for sandboxed, near-native execution in web browsers        |
| <b>WFP</b>             | Windows Filtering Platform -- kernel-mode framework for network packet filtering and inspection      |
| <b>WinTUN</b>          | Virtual TUN adapter driver for Windows, maintained by the WireGuard project                          |
| <b>WireGuard</b>       | Modern VPN protocol using Noise framework cryptography, known for simplicity and performance         |
| <b>XCFramework</b>     | Apple's binary framework format supporting multiple architectures in a single bundle                 |
| <b>Zeroize</b>         | Rust crate that securely clears memory contents (overwrites with zeros) when secrets go out of scope |

## 11.2 Document-Specific Abbreviations

| <b>Abbreviation</b> | <b>Full Form</b>                | <b>Context</b>                            |
|---------------------|---------------------------------|-------------------------------------------|
| <b>MVP2</b>         | Minimum Viable Product, Phase 2 | This project phase -- client applications |



| Abbreviation       | Full Form                                      | Context                                      |
|--------------------|------------------------------------------------|----------------------------------------------|
| <b>helix-core</b>  | Helix VPN Shared Rust Core                     | The libhelix_core Rust library               |
| <b>FRB</b>         | flutter_rust_bridge                            | Mobile Rust-Dart binding technology          |
| <b>UDL</b>         | UniFFI Definition Language                     | Interface definition for UniFFI bindings     |
| <b>LOC</b>         | Lines of Code                                  | Code size metric                             |
| <b>CI/CD</b>       | Continuous Integration / Continuous Deployment | Automated build and deployment pipeline      |
| <b>OTA</b>         | Over-The-Air                                   | Wireless software update delivery            |
| <b>D-Bus</b>       | Desktop Bus                                    | Linux inter-process communication system     |
| <b>MTU</b>         | Maximum Transmission Unit                      | Largest packet size for network transmission |
| <b>RSS</b>         | Resident Set Size                              | Physical memory usage metric                 |
| <b>FPS</b>         | Frames Per Second                              | UI rendering performance metric              |
| <b>P0/P1/P2/P3</b> | Priority levels (0 = highest)                  | Platform prioritization tiers                |

---

*Document compiled: 2026-07-03 Based on architecture research from 25+ independent sources including official framework documentation, open-source VPN client analysis, and cross-platform development benchmarks Document set: MVP2 Architecture Specification v1.0 Next review: 2026-07-17*

---

**END OF DOCUMENT**